

Single Agent Systems & Agent Pipelines – Quiz

Name –Ashish Vankati Gije

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

Answer: A stateful directed graph is a workflow in which each step can remember information from previous steps and use it in the next step. It is made up of nodes (tasks) and edges (connections) that decide how the workflow moves from one task to another. Since it keeps track of the current state, it can make decisions, branch into different paths, or even repeat certain steps when needed.

A simple linear pipeline, on the other hand, follows a fixed sequence of steps from start to finish. It does not remember previous information or change its flow based on different situations. Every task is executed in the same order regardless of the input.

So, the main difference is that a stateful directed graph is flexible and dynamic, while a linear pipeline is fixed and straightforward. A stateful graph is more suitable for intelligent agent systems because it can adapt its workflow based on the current state and previous actions.

2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

Answer: In an agent workflow, nodes are the individual tasks or actions that the agent performs. Each node has a specific responsibility, such as analyzing a user's query, calling a tool, or generating a response. Edges are the connections between these nodes that control how information moves from one step to the next.

For example, a Calculator Tool can be a node that performs mathematical calculations. The connection that sends a user's calculation request from the

Query Analysis node to the Calculator Tool node is an edge. Together, nodes and edges organize the workflow and ensure that the agent follows the correct sequence of actions to complete a task.

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Answer: Conditional routing is the process of sending a user's query to the most suitable tool or action based on its intent. Instead of treating every query the same, the agent first identifies what the user wants and then selects the appropriate tool to handle it. This makes the agent more efficient and accurate.

A simple rule-based routing logic can be:

- If the query contains the word "calculate", send it to the Calculator Tool.
- If the query contains the word "keywords", send it to the Keyword Extraction Tool.
- For all other queries, send them to the General Response module.

This type of routing is simple to implement and helps ensure that each query is processed by the most appropriate tool.

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Answer: Cycles (or loops) are important in agent pipelines because they allow the agent to repeat a task until it is completed successfully. They are useful when a task may fail or require more than one attempt. Instead of stopping after the first failure, the agent can retry the operation, making the system more reliable and robust.

A common example is an API request. If the request fails because of a temporary network issue or server timeout, the agent can retry the request a few times before returning an error. This increases the chances of completing the task successfully without requiring the user to try again manually.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

Answer: A single-agent system can simulate multi-agent behavior by dividing its work into different roles or stages. For example, it can first analyze the user's query, then decide which tool to use, and finally generate the response. Although these tasks seem like they are performed by different agents, they are all handled by the same system.

This approach makes the agent more organized and flexible without the complexity of managing multiple agents. As a result, a single-agent system can perform tasks in a way that is similar to a multi-agent architecture while remaining simpler to build and maintain.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

Answer: JSON schema tools define a standard format for the data exchanged between an agent and the tools it uses. They specify what fields are required, the data types, and the expected structure of the input and output. This ensures that the data is valid before it is processed.

Using JSON schemas makes the outputs consistent and easier to understand. It also reduces errors caused by missing or incorrect data and improves communication between different components of an agent system. As a result, agents and tools can work together more reliably and efficiently.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Answer: Sequential tool calls are performed one after another, where each tool depends on the result of the previous one. They are best used when the tasks are connected and must follow a specific order. For example, an agent may first analyze a user's query and then use the result to decide which tool to call next.

Parallel tool calls, on the other hand, run multiple independent tasks at the same time. They are useful when the tasks do not depend on each other. This helps reduce response time and improves overall efficiency.

In general, sequential tool calls are preferred for dependent tasks, while parallel tool calls are better for independent tasks that can be executed simultaneously.

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

Answer: Error handling helps an agent continue working even when something goes wrong. It makes the system more reliable and prevents it from crashing due to unexpected errors.

One common strategy is using try-except blocks to catch errors and return meaningful error messages instead of stopping the program. Another strategy is using a retry mechanism, where the agent automatically tries the operation again if it fails because of a temporary issue, such as a network error. In addition, logging errors is useful for tracking problems, debugging, and monitoring the system's performance.

These strategies improve the reliability of the agent and provide a better experience for users.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Answer: Trajectory evaluation is the process of examining the entire sequence of actions an agent takes to complete a task, rather than looking only at the final result. It includes checking the decisions made, the tools used, and the intermediate steps followed during execution.

This is important because an agent may produce the correct final answer while still making unnecessary or incorrect decisions along the way. By evaluating the complete trajectory, developers can identify hidden mistakes, improve the workflow, optimize tool usage, and make the agent more efficient and reliable.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Answer: Task completion rate is the percentage of tasks that an agent successfully completes. It shows how often the agent achieves the expected

result. Cost metrics measure the resources used while completing those tasks, such as API calls, processing time, or overall operational cost.

In a real-world system, these metrics can be measured by monitoring the agent's performance over many tasks. To improve them, developers can increase routing accuracy, avoid unnecessary tool calls, and use efficient algorithms. The goal is to achieve a high task completion rate while keeping the overall cost as low as possible, making the system both effective and efficient.